Self-Paced Module: Advanced Components and User Interaction

Name: _____
Section: _____
Date: _____

Java Programming · Grade 12 TVL · Cielito Zamora Senior High School
Week 19 · November 23–27, 2026 · Continues Week 18 Layouts and Panels

Day / time	Topic	Work and strip
Mon Nov 23 / 2 h	JComboBox	Page 1 Step 1; Monday strip, page 7 bottom.
Tue Nov 24 / 2 h	JCheckBox; radio group	Page 2 Step 2; Tuesday strip, page 7 middle.
Wed Nov 25 / 2 h	JList; combined inputs/actions	Page 3 Step 3; Wednesday strip, page 7 top.
Thu Nov 26 / 3 h	Safe dialogs; own-theme start	Pages 4–5 Step 4; task page 6; Thursday strip, page 8 bottom.
Fri Nov 27 / 3 h	Test, complete, present	Page 6 task; Friday strip, page 8 top.

How to use. Join both code columns into one file; work in order. Predict, run, correct and record results. LIPLET means quiz strip: submit only today's strip.

Offline: JDK only, Java 8 or later; all .java files in one folder. Compile `javac *.java`; `java Main` starts Vehicle; `java RequestDemo` starts practice. Close old windows before rebuilding. Error Guide: page 5.

Where we left off | Retain, add or replace

Retain unchanged: Vehicle, Jeepney, Tricycle, Bus, OverloadException, InvalidDistanceException and VehicleNotFoundException; DispatchThread, BoardingTask, ReadyReport, ConvoyLeg and LoadingBay; FareService, FareCheck and ReadyTask (all .java). Main.java remains the Week 18 launcher and readiness group.

Backup: copy Main.java to **Main_week18.txt** before any launcher edit. Retain older .txt backups; no second public Main in a .java file. **Replace:** TerminalPanel.java at Step 1. **Add/replace:** RequestDemo.java at Steps 2–4. Retain optional LayoutDemo, ThemeLayoutCheck and any ThemeCheck; they are not Vehicle.

Catch-up: use recovered Week 18 Main (Step 2) and TerminalPanel (Step 4) from CodeShare.

MONDAY | Choose One Plate: JComboBox

Definition. JComboBox is a drop-down for one choice; getSelectedItem reads it, or returns null for no selection [YORK-C]. Keep the nested panels, panels inside panels, from Week 18 [HWS].

> **ANALOGY** A clerk chooses a printed plate instead of typing it. A known plate still needs a valid distance.

Compare: JTextField accepts typed plate text; this JComboBox restricts plate choices. Distance stays a text field. Keep the service's unknown-plate, number and finite-positive checks; sample fares are classroom practice values, not LTFRB rates.

STEP 1 Choose a known Vehicle plate

Back up Main as above. Replace TerminalPanel.java only; compile `javac *.java`; run `java Main`. Retain all other files.

TerminalPanel.java — first column

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class TerminalPanel extends JPanel
    implements ActionListener {
  private JComboBox<String> plate = new
    JComboBox<>(
    new String[]{"NXA 123", "TRC 456", "BUS 789",
      "NYB 321"});
  private JTextField km = new JTextField("5", 5);
  private JLabel result = new JLabel(
    "Choose plate and km");
  public TerminalPanel() {
    setLayout(new BorderLayout(8, 8));
    plate.setSelectedIndex(-1);
    JLabel p = new JLabel("Plate:");
    JLabel d = new JLabel("Distance (km):");
    p.setLabelFor(plate); d.setLabelFor(km);
    JButton quote = new JButton("Quote fare");
    quote.setMnemonic('Q');
    quote.addActionListener(this);
    JPanel form = new JPanel(new GridLayout(2, 2,
      8, 8));
    form.add(p); form.add(plate);
    form.add(d); form.add(km);
    JPanel actions = new JPanel(new FlowLayout(
```

TerminalPanel.java — next column

```
      FlowLayout.LEFT));
    actions.add(quote);
    add(form, BorderLayout.NORTH);
    add(result, BorderLayout.CENTER);
    add(actions, BorderLayout.SOUTH);
  }
  @Override public void actionPerformed(
    ActionEvent e) {
    String chosen = (String) plate.getSelectedItem(
      );
    if (chosen == null) {
      result.setText("Choose a plate first.");
      return;
    }
    try {
      result.setText(FareService.quote(chosen,
        km.getText()));
    } catch (NumberFormatException ex) {
      result.setText("Enter a number for km.");
    } catch (VehicleNotFoundException ex) {
      result.setText(ex.getMessage());
    } catch (InvalidDistanceException ex) {
      result.setText(ex.getMessage());
    }
  }
}
```

Observed headless console: `No display: GUI not opened.` Window/dialog checks: page 5.

Meaning: <String> holds text choices; (String) reads this text value. Index -1 means no selection. The null guard stops before a fare call. EDT means event dispatch thread, the thread for interface work; Main.createContent still runs there.

Observed logic: no plate → Choose a plate first.; NXA 123 / abc → Enter a number for km.; corrected to 5 → NXA 123: P15.00. Run `java FareCheck` for the retained service tests.

References: [1 NTU], [2 HWS], [3 CMU], [4 NUS], [5 YORK-C]. Links/excerpts: page 5. **Key validated facts:** JComboBox reads one item or null [YORK-C]; each panel keeps its manager [NUS].

> **LIPLET · MONDAY** After Step 1 on page 1, answer the MONDAY strip (bottom of page 7) in pen. Detach only today's strip.

TUESDAY | Choose Independently or Exclusively

Definition. A JCheckBox is an independent on/off choice. A JRadioButton is one choice in a set; ButtonGroup makes the choices mutually exclusive, so selecting one clears the other. ButtonGroup controls selection; it is not a visible container. Add the buttons to a JPanel to arrange them [NTU] [BTN].

> **ANALOGY** At a school materials desk, a learner may need both a guide and a practice sheet, but chooses one place to collect them. The selection rule is different from the arrangement of the desk.

Choice type	Rule	In this example
JCheckBox	Each box is independent.	Guide, sheet, both or neither; Preview requires at least one.
JRadioButton + ButtonGroup	At most one in the group.	Desk or Classroom; Desk starts selected.

STEP 2 Choose materials independently and one collection place

Add RequestDemo.java, a separate school-materials practice program. Compile javac *.java; run java RequestDemo. Retain the Vehicle program; java Main still starts it. This example records choices only; it does not promise stock or submit a real request.

RequestDemo.java — first column

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class RequestDemo extends JPanel
    implements ActionListener {
  private JCheckBox guide = new JCheckBox(
    "Guide");
  private JCheckBox sheet = new JCheckBox(
    "Practice sheet");
  private JRadioButton desk = new JRadioButton(
    "Desk", true);
  private JRadioButton room = new JRadioButton(
    "Classroom");
  private JTextArea result = new JTextArea(4, 24);
  public RequestDemo() {
    setLayout(new BorderLayout(8, 8));
    setBorder(BorderFactory.createEmptyBorder(8, 8,
      8, 8));
    ButtonGroup collect = new ButtonGroup();
    collect.add(desk); collect.add(room);
    JPanel materials = new JPanel();
    materials.add(guide); materials.add(sheet);
    JPanel place = new JPanel();
    place.add(new JLabel("Collect at:"));
    place.add(desk); place.add(room);
    JPanel options = new JPanel(new GridLayout(2,
      1));
    options.add(materials); options.add(place);
    result.setEditable(false);
    result.setLineWrap(true);
      result.setWrapStyleWord(true);
    result.setText(
      "Choose materials, then Preview.");
    JPanel actions = new JPanel();
    for (String text : new String[]{"Preview"}) {
      JButton button = new JButton(text);
      button.setMnemonic(text.charAt(0));
      button.addActionListener(this);
      actions.add(button);
    }
    add(options, BorderLayout.NORTH);
    add(result, BorderLayout.CENTER);
    add(actions, BorderLayout.SOUTH);
```

RequestDemo.java — next column

```
  }
  String summary() {
    if (!guide.isSelected() && !sheet.isSelected(
      )) {
      throw new IllegalArgumentException(
        "Choose materials.");
    }
    if (!desk.isSelected() && !room.isSelected())
      {
      throw new IllegalArgumentException(
        "Choose a place.");
    }
    return "Guide=" + guide.isSelected()
      + "; Sheet=" + sheet.isSelected()
      + "; Place=" + (desk.isSelected() ? "Desk" :
        "Classroom");
  }
  @Override public void actionPerformed(
    ActionEvent e) {
    try { result.setText(summary()); }
    catch (IllegalArgumentException ex) {
      result.setText(ex.getMessage());
    }
  }
  public static void main(String[] args) {
    if (GraphicsEnvironment.isHeadless()) {
      System.out.println(
        "No display: GUI not opened.");
      return;
    }
    SwingUtilities.invokeLater(() -> {
        JFrame f = new JFrame(
          "School materials request");
        f.setDefaultCloseOperation(
          JFrame.DISPOSE_ON_CLOSE);
        f.setContentPane(new RequestDemo());
        f.setSize(640, 420);
        f.setMinimumSize(new Dimension(520, 380));
        f.setLocationRelativeTo(null);
        f.setVisible(true);
    });
  }
}
```

Observed headless console: `No display: GUI not opened.` Window/dialog checks: page 5.

Read the result. isSelected reads true/false; ?: chooses one of two texts. JTextArea is a multi-line text control, read-only here with wrapped feedback. The loop adds buttons; charAt(0) uses the first letter as a mnemonic (keyboard shortcut). The lambda () -> is a short Runnable for EDT startup.

Observed listener feedback

```
Guide=true; Sheet=true; Place=Desk
```

Both boxes + Desk. Checkbox combinations and exclusive-place logic passed headless.

> **Check yourself.** Preview with neither box, each alone and both. Alternate collection places: only one remains selected. Correct a missing-material error and Preview again.

Keyboard/display check: test Tab/Shift+Tab, Space, radio arrow keys and Alt+P on a school display. These physical actions remain unverified here.

References: [1 NTU], [2 HWS], [3 CMU], [4 NUS], [5 BTN]. Links/excerpts: page 5. **Key validated facts:** Checkboxes are independent; grouped radios are exclusive [BTN]. JPanel arranges visible controls [NUS].

> **LIPLET · TUESDAY** After Step 2 on page 2, answer the TUESDAY strip (middle of page 7) in pen. Detach only today's strip.

WEDNESDAY | Read a List and Combine Inputs

Definition. A JList displays rows of choices. SINGLE_SELECTION is a setting that allows at most one selected row. A JScrollPane is a container that provides scrolling when its child is larger than the visible area. Read getSelectedValue and check for null before using it [LIST].

ANALOGY A clerk combines subject, materials and one collection place before previewing a draft.

Read / action	Meaning
Preview / Clear	Validate and show the draft / reset its choices.

STEP 3 Combine a list selection with two different actions

Replace RequestDemo.java only; javac *.java, then java RequestDemo. Keep Vehicle. Compare Preview with Clear.

RequestDemo.java — first column

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class RequestDemo extends JPanel
    implements ActionListener {
  private JCheckBox guide = new JCheckBox(
    "Guide");
  private JCheckBox sheet = new JCheckBox(
    "Practice sheet");
  private JRadioButton desk = new JRadioButton(
    "Desk", true);
  private JRadioButton room = new JRadioButton(
    "Classroom");
  private JList<String> subject = new JList<>(
    new String[]{"Java", "English", "Math"});
  private JTextArea result = new JTextArea(4, 24);
  public RequestDemo() {
    setLayout(new BorderLayout(8, 8));
    setBorder(BorderFactory.createEmptyBorder(8, 8,
      8, 8));
    ButtonGroup collect = new ButtonGroup();
    collect.add(desk); collect.add(room);
    JPanel materials = new JPanel();
    materials.add(guide); materials.add(sheet);
    JPanel place = new JPanel();
    place.add(new JLabel("Collect at:"));
    place.add(desk); place.add(room);
    JPanel options = new JPanel(new GridLayout(2,
      1));
    options.add(materials); options.add(place);
    subject.setSelectionMode(
      ListSelectionModel.SINGLE_SELECTION);
    subject.setVisibleRowCount(3);
    JLabel label = new JLabel("Subject:");
    label.setLabelFor(subject);
    JPanel choices = new JPanel(new BorderLayout(4,
      4));
    choices.add(label, BorderLayout.NORTH);
    choices.add(new JScrollPane(subject),
      BorderLayout.CENTER);
    choices.add(options, BorderLayout.SOUTH);
    result.setEditable(false);
    result.setLineWrap(true);
      result.setWrapStyleWord(true);
    result.setText(
      "Choose materials, then Preview.");
    JPanel actions = new JPanel();
    for (String text : new String[]{"Preview",
      "Clear"}) {
      JButton button = new JButton(text);
      button.setMnemonic(text.charAt(0));
      button.addActionListener(this);
      actions.add(button);
    }
    add(choices, BorderLayout.NORTH);
    add(result, BorderLayout.CENTER);
    add(actions, BorderLayout.SOUTH);
  }
```

RequestDemo.java — next column

```
  String summary() {
    if (subject.getSelectedValue() == null) {
      throw new IllegalArgumentException(
        "Choose a subject.");
    }
    if (!guide.isSelected() && !sheet.isSelected(
      )) {
      throw new IllegalArgumentException(
        "Choose materials.");
    }
    if (!desk.isSelected() && !room.isSelected())
      {
      throw new IllegalArgumentException(
        "Choose a place.");
    }
    return subject.getSelectedValue() + ": Guide="
      + guide.isSelected()
      + "; Sheet=" + sheet.isSelected()
      + "; Place=" + (desk.isSelected() ? "Desk" :
        "Classroom");
  }
  void clearChoices() {
    subject.clearSelection();
    guide.setSelected(false); sheet.setSelected(
      false);
    desk.setSelected(true); result.setText(
      "Choices cleared.");
  }
  @Override public void actionPerformed(
    ActionEvent e) {
    if ("Clear".equals(e.getActionCommand())) {
      clearChoices(); return;
    }
    try { result.setText(summary()); }
    catch (IllegalArgumentException ex) {
      result.setText(ex.getMessage());
    }
  }
  public static void main(String[] args) {
    if (GraphicsEnvironment.isHeadless()) {
      System.out.println(
        "No display: GUI not opened.");
      return;
    }
    SwingUtilities.invokeLater(() -> {
        JFrame f = new JFrame(
          "School materials request");
        f.setDefaultCloseOperation(
          JFrame.DISPOSE_ON_CLOSE);
        f.setContentPane(new RequestDemo());
        f.setSize(640, 420);
        f.setMinimumSize(new Dimension(520, 380));
        f.setLocationRelativeTo(null);
        f.setVisible(true);
    });
  }
}
```

Observed headless console: `No display: GUI not opened.` Window/dialog checks: page 5.

Trace the actions: getActionCommand reads the button label. Clear resets before validation; Preview checks subject, materials and place. The catch allows correction. setVisibleRowCount(3) requests room for three rows; it does not limit data.

Observed listener feedback: empty → Choose a subject.; Java only → Choose materials.; Java + Guide → Java: Guide=true; Sheet=false; Place=Desk. Clear → Choices cleared.; English + Guide recovered.

Check yourself. Select Math and a sheet; verify the changed summary. Clear, then build another valid request. Test single-row selection and keyboard arrows on a school display.

References: [1 NTU], [2 HWS], [3 CMU], [4 NUS], [5 LIST]. Links/excerpts: page 5. **Key validated facts:** JList permits one row; JScrollPane scrolls its child [LIST]. Panels group actions [HWS].

LIPLET · WEDNESDAY After Step 3 on page 3, answer the WEDNESDAY strip (top of page 7) in pen. Detach only today's strip.

THURSDAY | Ask, Inform and Confirm Safely

Definition. A JOptionPane provides a ready-made dialog, a small window that asks or tells the user something. Information reports a result; input asks for text; confirmation asks whether to proceed [NTU]. An input cancellation returns null; a closed confirmation returns CLOSED_OPTION [YORK-D].

> **ANALOGY** A clerk can show a request, ask what to call it, or ask permission before clearing it. Silence, Cancel, No and closing a window do not mean permission.

STEP 4 Ask, inform and confirm safely

Replace only RequestDemo.java with this complete version. Compile javac *.java; run java RequestDemo. Info displays a valid request, Rename asks for a name, and Clear now requires Yes. Retain the Vehicle program and existing backups.

Reading order: page 4 left column, page 4 right column, then page 5 left and right. They form one RequestDemo.java. The final methods and launcher continue on page 5.

RequestDemo.java — first column

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class RequestDemo extends JPanel
    implements ActionListener {
  private JCheckBox guide = new JCheckBox(
    "Guide");
  private JCheckBox sheet = new JCheckBox(
    "Practice sheet");
  private JRadioButton desk = new JRadioButton(
    "Desk", true);
  private JRadioButton room = new JRadioButton(
    "Classroom");
  private JList<String> subject = new JList<>(
    new String[]{"Java", "English", "Math"});
  private String name = "My request";
  private JTextArea result = new JTextArea(4, 24);
  public RequestDemo() {
    setLayout(new BorderLayout(8, 8));
    setBorder(BorderFactory.createEmptyBorder(8, 8,
      8, 8));
    ButtonGroup collect = new ButtonGroup();
    collect.add(desk); collect.add(room);
    JPanel materials = new JPanel();
    materials.add(guide); materials.add(sheet);
    JPanel place = new JPanel();
    place.add(new JLabel("Collect at:"));
    place.add(desk); place.add(room);
    JPanel options = new JPanel(new GridLayout(2,
      1));
    options.add(materials); options.add(place);
    subject.setSelectionMode(
      ListSelectionModel.SINGLE_SELECTION);
    subject.setVisibleRowCount(3);
    JLabel label = new JLabel("Subject:");
    label.setLabelFor(subject);
    JPanel choices = new JPanel(new BorderLayout(4,
      4));
    choices.add(label, BorderLayout.NORTH);
    choices.add(new JScrollPane(subject),
      BorderLayout.CENTER);
    choices.add(options, BorderLayout.SOUTH);
    result.setEditable(false);
    result.setLineWrap(true);
      result.setWrapStyleWord(true);
    result.setText(
      "Choose materials, then Preview.");
    JPanel actions = new JPanel();
    for (String text : new String[]{"Preview",
      "Clear", "Info", "Rename"}) {
      JButton button = new JButton(text);
      button.setMnemonic(text.charAt(0));
      button.addActionListener(this);
      actions.add(button);
    }
    add(choices, BorderLayout.NORTH);
    add(result, BorderLayout.CENTER);
    add(actions, BorderLayout.SOUTH);
  }
  String summary() {
    if (subject.getSelectedValue() == null) {
```

RequestDemo.java — next column

```
      throw new IllegalArgumentException(
        "Choose a subject.");
    }
    if (!guide.isSelected() && !sheet.isSelected(
      )) {
      throw new IllegalArgumentException(
        "Choose materials.");
    }
    if (!desk.isSelected() && !room.isSelected())
      {
      throw new IllegalArgumentException(
        "Choose a place.");
    }
    return subject.getSelectedValue() + ": Guide="
      + guide.isSelected()
      + "; Sheet=" + sheet.isSelected()
      + "; Place=" + (desk.isSelected() ? "Desk" :
        "Classroom");
  }
  void clearChoices() {
    subject.clearSelection();
    guide.setSelected(false); sheet.setSelected(
      false);
    desk.setSelected(true); result.setText(
      "Choices cleared.");
  }
  void acceptName(String input) {
    if (input == null) { return; }
    String clean = input.trim();
    if (clean.isEmpty() || clean.length() > 30) {
      result.setText(
        "Use 1-30 characters for the name.");
      return;
    }
    name = clean; result.setText("Named: " +
      name);
  }
  void clearIfConfirmed(int answer) {
    if (answer == JOptionPane.YES_OPTION) {
      clearChoices();
    } else if (answer == JOptionPane.NO_OPTION) {
      result.setText("Kept choices (No).");
    } else if (answer ==
      JOptionPane.CANCEL_OPTION) {
      result.setText("Kept choices (Cancel).");
    } else {
      result.setText("Kept choices (Closed).");
    }
  }
  @Override public void actionPerformed(
    ActionEvent e) {
    String command = e.getActionCommand();
    if ("Rename".equals(command)) {
      acceptName(JOptionPane.showInputDialog(this,
        "Request name (1-30 characters):", name));
      return;
    }
    if ("Clear".equals(command)) {
      clearIfConfirmed(
        JOptionPane.showConfirmDialog(this,
        "Clear subject and materials?",
```

Safe state changes: acceptName checks null before trim; blank and over-30-character labels leave the previous name unchanged. The short-name limit keeps the request label readable. clearIfConfirmed clears subject/material choices only for Yes and retains the request name. Non-Yes outcomes report what happened.

Modal: a modal dialog waits for a response and can prevent interaction with its owning window until dismissed. That is expected. It differs from slow work freezing the interface. Keep sleep, join, await and slow file work out of listeners; retain ReadyTask's background work and safe updates.

Next: finish this file on page 5, run its checks, then show the teacher your own-theme plan on page 6. Use Thursday's last 60 minutes to build.

THURSDAY continued | Finish Step 4

RequestDemo.java — continued, first column

```
        "Confirm clear",
          JOptionPane.YES_NO_CANCEL_OPTION));
      return;
    }
    try {
      String text = name + "\n" + summary();
      if ("Info".equals(command)) {
        JOptionPane.showMessageDialog(this, text,
          "Request information",
            JOptionPane.INFORMATION_MESSAGE);
      } else {
        result.setText(text);
      }
    } catch (IllegalArgumentException ex) {
      result.setText(ex.getMessage());
    }
  }
  public static void main(String[] args) {
```

RequestDemo.java — next column

```
    if (GraphicsEnvironment.isHeadless()) {
      System.out.println(
        "No display: GUI not opened.");
      return;
    }
    SwingUtilities.invokeLater(() -> {
        JFrame f = new JFrame(
          "School materials request");
        f.setDefaultCloseOperation(
          JFrame.DISPOSE_ON_CLOSE);
        f.setContentPane(new RequestDemo());
        f.setSize(640, 420);
        f.setMinimumSize(new Dimension(520, 380));
        f.setLocationRelativeTo(null);
        f.setVisible(true);
    });
  }
}
```

Observed headless console: `No display: GUI not opened.` Window/dialog checks: page 5.

Dialog purpose	Call / safe outcome
Information	showMessageDialog: Info shows a valid summary. OK/close dismisses it without changing draft data.
Input	showInputDialog: text is accepted only after validation. Cancel/close returns null; acceptName returns before any change.
Confirmation	showConfirmDialog: Yes clears choices; No, Cancel and Closed each keep them. this identifies the owning panel.

Observed branch results: acceptName(" Lab practice ") stored Lab practice; null preserved it; blank/31 characters gave Use 1-30 characters for the name. No/Cancel/Closed kept Java + sheet; Yes cleared both and the list. These were direct method tests, not live dialog interactions.

Check yourself: try Info; Rename with valid/blank/31-character text, Cancel and close; then Clear with Yes, No, Cancel and close. Record the name and selections before/after each. Only Yes may clear the choices.

Error Guide | Actual compiler/service excerpts

Actual message excerpt	Meaning	Fix
error: cannot find symbol; symbol: class JCheckBox	Missing Swing import or wrong spelling.	Use import javax.swing.*; and the printed class name.
error: cannot find symbol; symbol: method getText()	A JComboBox is being read as a text field.	Use getSelectedItem; check null before processing.
NumberFormatException: For input string: "abc"	Distance is not a number.	Keep the number catch; correct km and try again.
InvalidDistanceException: Use a finite km above 0.	Zero, negative, NaN or infinity.	Use a finite positive km; keep its separate catch.
VehicleNotFoundException: Unknown plate: ZZZ	The service received an unknown plate.	Retain service validation even with restricted choices.

Produced with JDK 21 and Java 8 compatibility compilation; compiler line numbers may differ. A separate display probe returned `java.awt.AWTError: Can't connect to X11 window server` for DISPLAY :0; no virtual display was installed.

What was tested | What to check on a school display

Compilation: all four printed versions passed Java 8 compilation and headless launch. **Program logic:** controls/listeners, checkbox states, radio exclusivity, list/actions, recovery, dialog-result branches and readiness completion/interruption passed. **Window appearance:** not observed. **Resizing:** live resizing not observed. **User actions:** listener/method calls passed; physical clicks, keys and actual dialog acceptance/cancellation/closing were not observed.

School checks: run both launchers; test missing/valid/invalid/corrected input, every dialog outcome and unchanged data after cancellation. Resize Vehicle at 520×300 and 720×320, RequestDemo at 520×380, 640×420 and larger. Test long feedback, labels, Tab/Shift+Tab, Space, arrows, Alt shortcuts and fare during readiness. Record failures too.

Five opened university references per day

NTU: Nanyang Technological University; HWS: Hobart and William Smith Colleges; CMU: Carnegie Mellon University; NUS: National University of Singapore; YORK: York University.

[NTU] GUI Programming Part 2. Excerpt: “exclusive selection”. Open source

[HWS] Basic Layout. Excerpt: “preferred size”. Open source

[CMU] Using Layout Managers. Excerpt: “layout manager”. Open source

[NUS] How to Use Panels. Excerpt: “Adding Components”. Open source

[YORK-C] JComboBox. Excerpt: “current selected item”. Open source

[BTN] Buttons, Check Boxes and Radio Buttons. Excerpt: “previously selected button”. Open source

[LIST] How to Use Lists. Excerpt: “Only one item”. Open source

[YORK-D] JOptionPane. Excerpt: “CLOSED_OPTION”. Open source

Daily numbered sets: Mon: 1 NTU, 2 HWS, 3 CMU, 4 NUS, 5 YORK-C. Tue: 1 NTU, 2 HWS, 3 CMU, 4 NUS, 5 BTN. Wed: 1 NTU, 2 HWS, 3 CMU, 4 NUS, 5 LIST. Thu/Fri: 1 NTU, 2 HWS, 3 CMU, 4 NUS, 5 YORK-D. CMU/NUS host older tutorial text: use this module’s Java 8 code and safe outcome checks.

References: [1 NTU], [2 HWS], [3 CMU], [4 NUS], [5 YORK-D]. Links/excerpts: page 5. **Key validated facts:** Cancelled input returns null; closed confirmation returns CLOSED_OPTION [YORK-D]. Only explicit Yes clears choices.

LIPLET · THURSDAY After Step 4 on page 5, answer the THURSDAY strip (bottom of page 8) in pen. Detach only today’s strip.

FRIDAY | Your Own Theme: Interaction Task and Presentation

Definition. An interaction is useful when a user can choose, act, understand the result and recover. Extend your own theme from Weeks 11–12 with purposeful controls, working model calls and honest tests. Keep useful nested panels and suitable layout managers [HWS] [NUS].

> **ANALOGY** Arrange your own store, canteen, pet-care or school-service counter around what its user needs. Show what the user can safely change and what needs confirmation.

Start Thursday after Step 4; present Friday. Use your own theme, rather than copying Vehicle or submitting RequestDemo as your own work. Learn all four selection-control types and three dialog purposes; use only those that serve your task. Require at least two types and one appropriate dialog.

Requirements checklist

☐ Extend your own theme; keep model rules and useful workers.

☐ Use at least two selection-control types with clear purposes.

☐ Combine inputs; reject missing, blank or invalid values.

☐ Provide two different useful actions with working listeners.

☐ Use at least one appropriate dialog.

☐ Test every offered outcome; keep data on cancellation.

☐ For input check null; for confirmation act only on Yes.

☐ Keep nested panels, suitable managers, labels and feedback.

☐ Use EDT for controls; keep slow work off listeners.

☐ Record tests, complete self-check and present your own code.

Five-minute presentation

• Introduce your theme and intended user (30 s).

• Run one normal combined-input action (1 min).

• Prove bad input, recovery, a different action and safe dialog outcomes (2 min).

• Point to two controls, nested groups, a model call and the handler (1 min).

• Answer a question about your own code (30 s).

Plan before typing | Teacher checks first

Plan item	Your decision
Theme / intended user	_____
Existing model + useful call	_____
Two control types / why	_____ _____
Combined inputs / rules	_____
Two actions / dialog purpose	_____ _____
Cancel/No/close behavior	_____
Nested groups / worker	_____

Testing evidence | Exact inputs and outcomes

Test / expected	Actual / pass or fix
Normal combined input	_____
Missing / blank / invalid	_____
Correction / recovery	_____
Different useful action	_____
Dialog accept / cancel / close	_____
Each confirm result, if used	_____
Resize / longest feedback	_____
Keyboard / responsiveness	_____

Record actual outcomes. If a check was unavailable, write “not tested” and explain. Include a failed test and the repair. Keep existing slow work off listeners; if your theme needs no worker, explain why.

Two three-hour days

Thursday: 100 min lesson/checks/strip; 20 min plan; 60 min development. **Friday:** 10 min recall; 50 min finish; 30 min tests/repair; 15 min rehearsal; 60 min presentations; 15 min slip/reflection.

For a large class: use parallel circles of up to 12, peer records and teacher rotation.

A tested example to adapt, not submit

Pages 2–5 show a tested school-service theme. A canteen could instead use an item drop-down and an independent “pack separately” box, with Preview and confirmed Clear. Retain its existing model rules; no new price or discount is needed.

Starter fragment | Fill the blanks

```
JComboBox<String> choice =
  new JComboBox<>(___);
JCheckBox extra = new JCheckBox("___");
JPanel form =
  new JPanel(new ___(2, 2, 8, 8));
String input = JOptionPane.showInputDialog(
  form, "___");
if (input == ___) { return; }
// Validate input before changing your model.
```

Fragment only, inside an existing method/listener. Fill choices/text, GridLayout and null; add labelled inputs and validate before a useful model call. Keep working model code.

Self-check before presenting

Check	Yes / Not yet
Two purposeful control types + groups	Yes / Not yet
Combined input + two useful actions	Yes / Not yet
Safe dialog + recovery + model rules	Yes / Not yet
EDT, keyboard, resize and test record	Yes / Not yet

Prepare using your own code

1. Why do these two selection-control types suit your user?
2. Where do combined inputs reach your existing model?
3. What happens after Cancel, No or closing, where offered?
4. Which test failed, what changed, and what passed afterward?
5. How do your layout and thread choices keep the interface usable?

Scoring rubric | 50 points

Criterion	Pts	Score
Purposeful selection controls and nested groups	10	___
Combined inputs and two useful actions	10	___
Safe dialog outcomes and validation/recovery	10	___
Retained model, EDT and useful workers	8	___
Actual test evidence and accessibility	6	___
Presentation and own-code explanation	6	___
TOTAL	50	___

References: [1 NTU], [2 HWS], [3 CMU], [4 NUS], [5 YORK-D]. Links/excerpts: page 5. **Key validated facts:** Nested panels organize work [HWS]; dialog results distinguish a choice from cancellation or closing [YORK-D].

> **LIPLET · FRIDAY** After Step 4 on page 5 and your own-theme presentation, answer the FRIDAY strip (top of page 8) in pen. Detach only today’s strip.

WEDNESDAY · NOV 25 Read a List and Combine Inputs

Answer after Step 3 on page 3.

Name: _____ Section: _____ Score: _____ / 15

3.1 Define (5 pts): In Step 3, define JList, JScrollPane and SINGLE_SELECTION.

3.2 Explain (5 pts): In Step 3, trace a Preview action through the subject, material and place checks. How does Clear differ?

3.3 Connect (5 pts): Connect Step 3 to Week 13: how does catching invalid input allow correction and another Preview without restarting?

CUT ON TUESDAY · DETACH THE TUESDAY STRIP BELOW

TUESDAY · NOV 24 Choose Independently or Exclusively

Answer after Step 2 on page 2.

Name: _____ Section: _____ Score: _____ / 20

2.1 Define (5 pts): In Step 2, how does a JCheckBox differ from a JRadioButton in a ButtonGroup?

2.2 Explain (5 pts): In Step 2, why must the radio buttons be added to both ButtonGroup and a visible JPanel?

2.3 Identify (5 pts): For Step 2, fill four blanks: (a) two materials at once: ___; (b) one place: ___; (c) invisible selection coordinator: ___; (d) method that reads on/off state: ___.

2.4 Connect (5 pts): Connect Step 2 to Week 18: how do the materials, place and action panels keep choices easy to find?

CUT ON MONDAY · DETACH THE MONDAY STRIP BELOW

MONDAY · NOV 23 Choose One Plate: JComboBox

Answer after Step 1 on page 1.

Name: _____ Section: _____ Score: _____ / 15

1.1 Define (5 pts): In Step 1, what does JComboBox do, and what does a null selected item mean?

1.2 Explain (5 pts): In Step 1, why does replacing the plate field not remove distance validation or the three exception catches?

1.3 Connect (5 pts): Connect Step 1 to Weeks 12–13: which model/service rules stay separate from the interface choice?

FRIDAY · NOV 27 Your Own Theme: Interaction Task and Presentation

Answer after Step 4 on page 5 and your presentation.

Name: _____ Section: _____ Score: _____ / 20

5.1 Point (5 pts): In your own code, point to two selection-control types, the combined-input check, two useful actions and your dialog handler.

5.2 Evaluate (5 pts): Defend a control and dialog choice in your own program. Use your cancellation or recovery test as evidence.

5.3 Apply (5 pts): A school office needs several independent document choices and one collection place. Adapt your own theme code: name the controls, validation, useful actions and safe dialog outcome.

5.4 Reflect (5 pts): Compare your own program before and after Week 19. Describe one failed test, its repair, the observed new result and one remaining improvement.

CUT ON THURSDAY · DETACH THE THURSDAY STRIP BELOW

THURSDAY · NOV 26 Ask, Inform and Confirm Safely

Answer after Step 4 on page 5.

Name: _____ Section: _____ Score: _____ / 15

4.1 Define (5 pts): In Step 4, name the three dialog purposes and explain what modal means.

4.2 Explain (5 pts): In Step 4, why must acceptName test null before trim, and why may only YES_OPTION clear choices?

4.3 Connect (5 pts): Connect Step 4 to Weeks 16–17: how does a modal dialog differ from slow work freezing the event dispatch thread?

Week 19 CodeShare practice additions

Use this appendix with the original module. Original lesson pages and sample listings remain unchanged. Text backups and data remain unchanged in the supporting-files folder.

Console checks exercise component state, layout or event behavior without opening a window. The message No display: GUI not opened. alone never completes an alteration. Check the actual interface on a school computer; browser GUI rendering and interaction remain unverified.

Run the unchanged sample first. Sample verified records the demonstration only. Make Your task changes, then compare the complete Expected task output. A successful alteration earns Completed. Keep module work, personal work and submitted versions separate. Paper liplets remain teacher assessed.

Step 1 alteration

Sample: java-2-w19-d1-s1 Task: java-2-w19-d1-s1-alter-v1

Initially select TRC 456 at index 1 instead of leaving the combo box unselected. Invoke the existing quote handler and print its actual result.

Before the existing headless check in `Main.java`, add this console check. Keep the original sample message and the window code.

```
TerminalPanel panel = new TerminalPanel();
panel.actionPerformed(null);
javax.swing.JLabel result = (javax.swing.JLabel) ((java.awt.BorderLayout)
panel.getLayout()).getLayoutComponent(java.awt.BorderLayout.CENTER);
System.out.println(result.getText());
```

Expected task output

```
TRC 456: P70.00
No display: GUI not opened.
```

Step 2 alteration

Sample: java-2-w19-d2-s2 Task: java-2-w19-d2-s2-alter-v1

Make Classroom the initial collection place after creating the radio-button group. For the check, select Guide and print summary() from that same RequestDemo instance.

Before the existing headless check in `RequestDemo.java`, add this console check. Keep the original sample message and the window code.

```
RequestDemo demo = new RequestDemo();
demo.guide.setSelected(true);
System.out.println(demo.summary());
```

Expected task output

```
Guide=true; Sheet=false; Place=Classroom
No display: GUI not opened.
```

Step 3 alteration

Sample: java-2-w19-d3-s3 Task: java-2-w19-d3-s3-alter-v1

Change clearChoices() so Classroom is selected after clearing. For the check, clear a request, select Math and Practice sheet, then print its actual summary.

Before the existing headless check in `RequestDemo.java`, add this console check. Keep the original sample message and the window code.

```
RequestDemo demo = new RequestDemo();
demo.clearChoices();
demo.subject.setSelectedIndex(2);
demo.sheet.setSelected(true);
System.out.println(demo.summary());
```

Expected task output

```
Math: Guide=false; Sheet=true; Place=Classroom
No display: GUI not opened.
```

Step 4 alteration

Sample: java-2-w19-d4-s4 Task: java-2-w19-d4-s4-alter-v1

Reduce the accepted request-name limit from 30 to 12 characters and update the corresponding validation message. Try "Classroom materials". Print the rejection and unchanged stored name.

Before the existing headless check in `RequestDemo.java`, add this console check. Keep the original sample message and the window code.

```
RequestDemo demo = new RequestDemo();
demo.acceptName("Classroom materials");
System.out.println(demo.result.getText());
System.out.println("Name retained: " + demo.name);
```

Expected task output

```
Use 1-12 characters for the name.
Name retained: My request
No display: GUI not opened.
```